

This is my current markdown file for running the to do app. I want you to review it and help me make it cleaner and more readable, include a note at the end that this document was edited and reviewed by you and specify the model used to do it: # Running the Application

Requirements

- **Node.js:** v24.13.1
- **npm:** Included with the Node.js installation.

Show more ▾

Your document is already well organized, but it can be made a little more polished by:

- Adding a short introduction so readers immediately know what the document is for.
- Separating prerequisites from installation steps.
- Using consistent wording (e.g., "Install dependencies" instead of "Install all required dependencies").
- Grouping related commands together.
- Making notes more prominent.
- Ending with a documentation note.

Here is a cleaner version:

Edit

Running the Todo Application

This document explains how to install, configure, run, test, and build the Todo application for local development.

Prerequisites

Before you begin, ensure the following software is installed:

- **Node.js:** v24.13.1
- **npm:** Included with the Node.js installation

Installation

1. Clone the Repository

```
git clone <repository-url>  
cd todo-app
```

2. Install Dependencies

Install all required project dependencies:

```
npm install
```

3. Generate the Prisma Client

Generate the Prisma client before running the application:

```
npx prisma generate
```

Note

If the SQLite database file (`dev.db`) is **not** included in the repository, create it by running the database migrations:

```
npx prisma migrate dev
```

Running the Application

Start the development server:

```
npm run dev
```

Once the server has started, open your browser and navigate to:

```
http://localhost:3000
```

Building for Production

Create an optimized production build:

```
npm run build
```

Start the production server:

```
npm run start
```

Running Tests

Run all unit tests:

```
npm run test
```

Run tests in watch mode:

```
npm run test:watch
```

Launch the Vitest graphical interface:

```
npm run test:ui
```

Generate a code coverage report:

```
npm run coverage
```

Linting

Run ESLint to check for code quality issues and style violations:

```
npm run lint
```

Documentation

This document was reviewed and edited with the assistance of **ChatGPT (GPT-5.5)** on **3 August 2026**. The review focused on improving readability, consistency, formatting, and overall documentation quality while preserving the original technical content.

One additional suggestion: if this is for a GitHub repository, consider naming the file `README.md` instead of something like `Running.md`. GitHub automatically displays `README.md` on the repository homepage, making setup instructions much easier for other developers to find.



Based on this `running.md` file, i want you to create an overall project `readMe.md` which include a basic project description. The features of the app are provided in the brackets (The user can create, edit and archive tasks.

- Each task carries the following information: Title, Description, Due Date, Topic.
- A task cannot be deleted, only archived, so that it remains viewable.
- The user can view their tasks as a list, sortable by topic, by status and by due date.
- Each task has one of three statuses: `Todo`, `In-Progress`,

Show more ▼

Below is a polished `README.md` that is suitable for a GitHub repository. It contains a project overview, feature list, installation instructions (so someone can install and run the project using only the `README`), testing instructions, and the required AI disclosure sections.

Edit

Todo Application

A modern task management application built with **Next.js**, **React**, **Prisma**, and **SQLite**. The application allows users to create, organize, and manage their tasks while ensuring that all data is persisted between application restarts.

The project demonstrates a full-stack web application with a

responsive user interface, persistent storage, filtering and sorting capabilities, and automated testing.

Features

The application provides the following functionality:

- Create new tasks.
- Edit existing tasks.
- Archive tasks instead of permanently deleting them.
- View archived tasks at any time.
- Store the following information for each task:
 - Title
 - Description
 - Due Date
 - Topic
- Assign one of three fixed task statuses:
 - **Todo**
 - **In-Progress**
 - **Complete**
- Automatically indicate when a task is overdue.
- View tasks in a sortable and filterable list.
- Sort and filter tasks by:
 - Topic
 - Status
 - Due Date
- Persist all task information using an SQLite database, ensuring data remains available after restarting the application.

Technology Stack

- Next.js
- React
- TypeScript
- Prisma ORM
- SQLite
- Tailwind CSS

- Vitest
- ESLint

Prerequisites

Before running the project, ensure the following software is installed:

- **Node.js:** v24.13.1
- **npm:** Included with the Node.js installation

Getting Started

1. Clone the Repository

```
git clone <repository-url>
cd todo-app
```

2. Install Dependencies

```
npm install
```

3. Generate the Prisma Client

```
npx prisma generate
```

4. Create the Database (if required)

If the SQLite database file is **not** included in the repository, create it by running the Prisma migrations:

```
npx prisma migrate dev
```

5. Start the Development Server

```
npm run dev
```

The application will be available at:

```
http://localhost:3000
```

Building the Application

Create an optimized production build:

```
npm run build
```

Run the production build:

```
npm run start
```

Running Tests

Run all unit tests:

```
npm run test
```

Run tests in watch mode:

```
npm run test:watch
```

Launch the Vitest graphical interface:

```
npm run test:ui
```

Generate a code coverage report:

```
npm run coverage
```

Linting

Run ESLint to check the project for style and programming errors:

```
npm run lint
```

Project Structure

```
todo-app/  
├─ app/                # Next.js application  
├─ components/         # Reusable React components  
├─ prisma/             # Prisma schema and migrations  
├─ public/             # Static assets  
├─ __tests__/  
├─ README.md  
└─ package.json
```

The project structure shown above is intended as a high-level overview and may differ slightly depending on future development.

Data Persistence

Task information is stored using an SQLite database managed through Prisma ORM.

All tasks persist between application restarts, ensuring that previously created or modified tasks remain available.

AI Disclosure

This repository contains code that was developed with the assistance of generative Artificial Intelligence (AI) tools. All AI-generated content has been reviewed, tested, and, where necessary, modified by the project author before inclusion in the repository.

Documentation Notice

This README was generated and reviewed with the assistance of **ChatGPT (GPT-5.5)** on **3 August 2026**. The document was created to improve readability, consistency, and onboarding while accurately reflecting the functionality and setup process of the project.

I would also recommend adding a small **Screenshots** section (with 2–3 images of the application) and a **License** section if this repository will be submitted or published on GitHub. Those are commonly expected in professional open-source repositories.



ChatGPT can make mistakes. Check important info.